Intelligent External Hardware Monitor With AI Diagnostics

Jason Kostopoulos
Unisversity of Michigan - Dearborn
Dearborn, MI
jasok@umich.edu

Brad Bernia
Unisversity of Michigan - Dearborn
Dearborn, MI

Ryan Chang
Unisversity of Michigan - Dearborn
Dearborn, MI

Abstract*—Traditional consumer-grade PC hardware monitoring tools rely on the host operating system's graphics stack, rendering them obsolete the moment the system crashes, freezes or when resource usage is too extreme. This is exactly when diagnostic data is most critical and so this project presents an external hardware monitoring system to provide crash resilient telemetry visualization and intelligent root-cause analysis. The architecture of this project consists of a host daemon that extracts system resources every second and transmits them via UART communication to a separate Raspberry Pi module. To ensure independence from the host OS, the Raspberry Pi operates a custom bare-metal C Real-Time Operating System (RTOS), utilizing hardware interrupts and direct GPU mailbox memory-mapped I/O for independent UI rendering. Furthermore, the host system maintains a rolling 60-second log. Upon detecting critical resource anomalies, the system queries a Large Language Model (LLM) via Cloud API to generate an automated, human-readable diagnostic report, which is forwarded to the external Raspberry Pi module. The resulting system bridges the gap between expensive enterprise Out-Of-Band Baseboard Management Controllers (BMC) and standard consumer monitors, offering a low-cost, reliable diagnostic tool augmented with AI-Driven anomaly detection.

Keywords—AIOps, Anomaly Detection, Bare-Metal Programming, Large Language Models, System Telemetry, UART, Fault Tolerance

I. Introduction

A. Literature Survey

Traditional hardware monitoring tools rely heavily on the host operating system, not only to collect diagnostic data but also to render the visual display. While host-side daemons are efficient for data extraction, previous literature indicates that in-band monitoring innately introduces system overhead and can be impractical for lightweight, nonstop telemetry [1], [2]. A bigger issue is that relying on the host's graphics stack for visualization creates a critical single point of failure. As highlighted in enterprise studies, when a hardware fault or kernel panic occurs, the host operating system halts [3]. This also terminates the monitor, causing the UI to freeze or fail entirely. Recent research defines this phenomenon as a structural telemetry collapse, where system instability simultaneously degrades the monitoring payload and causes a complete disappearance of device metrics [4].

Because of this, the user is left blind, with no on-screen access to the critical telemetry or system logs that preceded the crash. To figure out exactly why these crashes happen, modern systems are increasingly shifting toward automated, machine learning powered anomaly detection to parse system data independently of host rendering [5]. While the structural collapse of host-based telemetry is extensively documented in enterprise servers and high performance computing (HPC) environments [1], [3], these identical architectural vulnerabilities exist in consumer grade PCs.

B. State of the Art

To address the vulnerabilities discussed above, modern state of the art enterprise architectures have adopted excludively Out-of-Band Management as the industry standard. This is primarily achieved through Baseboard Management Controllers (BCMs), which are independent hardware modules that operate on a completely decoupled execution environment. They utilize a separate processor and memory pool and they ensure continuous diagnostic monitoring, allowing administrators to monitor hardware states even if during total host failure [6].

While Out-of-Band hardware resolves the problem of availability, analyzing the resulting raw statistics and crash logs can be a daunting task and traditionally has required manual intervention. Recently, the state of the art in system diagnostics has shifted towards Artificial Intelligence for IT Operations (AIOps). As shown by recent studies on rapid outage resolution, Large Language Models are no utilizes for highly accurate and automated root cause analysis [7]. Additionally, modern LLM frameworks have proven capable of performing human-free anomaly detection by autonomously parsing complex, unstructured system logs and data [8]. By combinging decoupled out-of-band hardware with LLM-powered log analysis, state of the art infrastructure achieves both high fault tolerance and intelligent anomaly resolution.

C. Proposed Solution

Our proposed solution scales these enterprise fault tolerance principles down to an accessible consumer level monitor. Our project is an external system monitor utilizing a Raspberry Pi, currently implemented via the **QEMU hardware emulator**. Unlike most traditional consumer-level hardware monitors, the visualization is executed entirely on the independent hardware, ensuring the telemetry display remains active even during a system lock-up or a host kernel panic. A lightweight Python daemon running on the host PC securely collects telemetry data and transmits it to the external monitor over UART. To mirror state of the art AIOps frameworks, the system integrates the Google Gemini Large Language Model API to autonomously parse hardware telemetry and crash logs. If the host PC experiences system anomalies (like high CPU temperatures, or high RAM usage), the Python daemon queries the LLM with a custom prompt and a 60 second rolling log of system data. The LLM generates a human-readable root cause analysis, which is then sent to the Raspberry Pi, by the Python daemon, to be displayed.

D. Organization of the Report

The remainder of the report is organized as follows: Section II details the project requirements we gathered in the project proposal and is the first phase of out waterfall documentation. Section III outlines the System Design. Section IV is the implementation where we've provided some snippets of code. Section V outlines the system evaluation and testing methods we used to validate our requirements. Section VI provides a small user manual for the emulating the bare metal kernel on the QEMU emulator and for the Python Daemon that is required to be installed on the host PC. And Section VII concludes our report.

II. Project Requirements (waterfall phase 1)

A. Overview

This section represents the Software Requirements Specification (SRS) for the Hardware RTOS PC Monitor, an embedded system consisting of a Windows host daemon and a Raspberry Pi bare-metal RTOS. The host daemon collects CPU, RAM, disk, and thermal metrics, can query the Google Gemini AI API for anomaly reports, and transmits structured data packets to the Pi over a UART link. The Pi firmware renders the received data on a connected display, evaluates alarm thresholds, supervises host connection with a software watchdog timer, and enters a Standby Mode when no host connection is present.

Requirements are identified by unique identifiers of the form SRS-NNN. Functional requirements define system behavior, including alarm thresholds, and displayed outputs. Non-functional requirements define performance, security, and power constraints. Timing requirements define real-time deadlines and are classified as Hard, Firm, or Soft real-time requirements.

B. Functional Requirements

The following requirements define the observable, behavioral capabilities of the system.

SRS-001:
The following information shall be updated once every 20ms:

- Monitored Temperature
- Monitored CPU usage
- Monitored Memory usage
- Monitored Disk usage

SRS-002:
The following information shall be displayed:

- Monitored temperature
- Monitored CPU usage
- Monitored Memory usage
- Monitored Disk usage

SRS-003:
When the option to generate an AI analyzed log is selected, the log shall be formatted in JSON and sent over the AI API.

SRS-004:
When an analyzed log is received from the AI API, the analyzed log shall be displayed.

SRS-005:
Monitored information shall be captured over UART connection.

SRS-006:
When the monitored CPU % becomes greater than 75%, the CPU HIGH WARNING alarm shall be triggered.

SRS-007:
When the monitored RAM % becomes greater than 85%, the RAM HIGH WARNING alarm shall be triggered.

SRS-008:
When the monitored Disk % becomes greater than 70%, the DISK HIGH WARNING alarm shall be triggered.

SRS-009:
When the monitored Temp becomes greater than 80 °C, the TEMP HIGH WARNING alarm shall be triggered.

SRS-010:
When the monitored CPU % becomes greater than 95%, the CPU CRITICAL WARNING alarm shall be triggered.

SRS-011:
When the monitored RAM % becomes greater than 95%, the RAM CRITICAL WARNING alarm shall be triggered.

SRS-012:
When the monitored Disk % becomes greater than 90%, the DISK CRITICAL WARNING alarm shall be triggered.

SRS-013:
When the monitored Temp becomes greater than 95 °C, the TEMP CRITICAL WARNING alarm shall be triggered.

SRS-014:
If the UART communication is lost unexpectedly, the COMMS DISCONNECT alarm shall be triggered.

SRS-015:
When a connection is detected over UART, Monitoring Mode shall be entered.

SRS-017:
On device power-up or device disconnect, Standby Mode shall be entered.

SRS-018:
If the watchdog timer is not serviced for 500ms, the system shall reset.

SRS-019:
When a critical warning alarm is triggered, an AI analyzed log shall be generated and displayed.

C. Alarm Table

The table below defines every alarm condition. These also constitute the anomalies the system looks for. Each alarm specifies the exact text string rendered in the display banner and the color of that banner.

TABLE I. ALARM TABLE

Alarm Name	Display Text	Banner Color	SRS Ref
CPU HIGH WARNING	[WRN] PC CPU LOAD HIGH: Monitoring Jitter Likely	Yellow	SRS -006
RAM HIGH WARNING	[WRN] PC RAM SATURATED: Background Tasks Slow	Yellow	SRS -007
DISK HIGH WARNING	[WRN] PC DISK BUSY: Data Logging Delayed	Yellow	SRS -008
TEMP HIGH WARNING	[WRN] PC THERMAL RISE: Check PC Airflow	Yellow	SRS -009
CPU CRITICAL WARNING	[CRT] PC CPU OVERLOAD: Monitoring Unreliable	Red	SRS -010
RAM CRITICAL WARNING	[CRT] PC RAM DEPLETED: System Instability Risk	Red	SRS -011
DISK CRITICAL WARNING	[CRT] PC DISK STALL: Log Write Failure	Red	SRS -012
TEMP CRITICAL WARNING	[CRT] PC THERMAL CRITICAL: Throttling Detected	Red	SRS -013
COMMS DISCONNECT	[FTL] PC LINK LOST: Check USB Connection/Host App	Red	SRS -014

D. Non-Functional Requirements

The following requirements define performance, security, and power constraints that the system must satisfy.

SRS-021:

AI API shall use TLS 1.3 encryption.

SRS-022:

The watchdog timer shall use an independent real clock.

SRS-023:

After the watchdog timer is reset, the system shall be fully operational within 500ms.

E. Timing Requirements

Timing requirements are classified by real-time category. Hard real-time deadlines must never be missed, as a miss constitutes a system failure. Firm real-time deadlines should not be missed. Late results are discarded and a visible degradation is acceptable. Soft real-time deadlines represent deadlines where occasional latency is tolerable.

a) Hard Real-Time Requirements

TABLE II. HARD REAL-TIME REQUIREMENTS

Task	Deadline	Justification
Data Retrieval (UART)	< 0.1 ms per byte	Byte must be read from UART before the next byte arrives at 115,200 baud to prevent FIFO overflow.
Scheduler Tick	1 ms	The system tick must be precise so that the Watchdog and Display timers do not drift over time.
Watchdog Timer	500 ms	If no data is received for 500 ms, the system must conclude that the Host PC has been disconnected or crashed.

b) Firm Real-Time Requirements

TABLE III. FIRM REAL-TIME REQUIREMENTS

Task	Deadline	Justification
Display Cycle	20 ms (50 Hz)	If the display update exceeds 20 ms, the frame should be dropped rather than delaying the next data packet. Some visual tearing is acceptable.
Anomaly Trigger/ Interrupt	< 50 ms from detection	Visual alarms (screen color change) must appear effectively instant to the human eye when a threshold is crossed.

c) Soft Real-Time Requirements

TABLE IV. SOFT REAL-TIME REQUIREMENTS

Task	Deadline	Justification
LLM Report Generation	< 3 seconds (target)	The user can wait a few seconds for a text explanation. The RTOS must not block critical tasks while waiting for this response.
System Boot	< 2 seconds (target)	The monitor should be active before the host PC finishes its boot sequence.

III. System Design (Waterfall Phase 2)

A. Overview

The Hardware RTOS PC Monitor is an external hardware monitoring system consisting of two independent parts: a Windows host daemon and a Raspberry Pi with a display. The host daemon polls OS-level performance metrics and transmits them to the Pi over a USB/UART link. The Pi operates on bare-metal firmware with no host operating system dependency, preserving the last known system state even if the host crashes or freezes.

The system continuously monitors CPU usage and temperature, RAM usage, and disk I/O usage. When any metric crosses a defined threshold, a prioritized alarm banner appears on the Pi's LCD display. The system supports two operating modes - Standby Mode (awaiting connection) and Monitoring Mode (actively receiving and displaying data) - and integrates an AI diagnostic component that generates root-cause analysis reports via the Google Gemini API.

B. System Architecture Flowchart

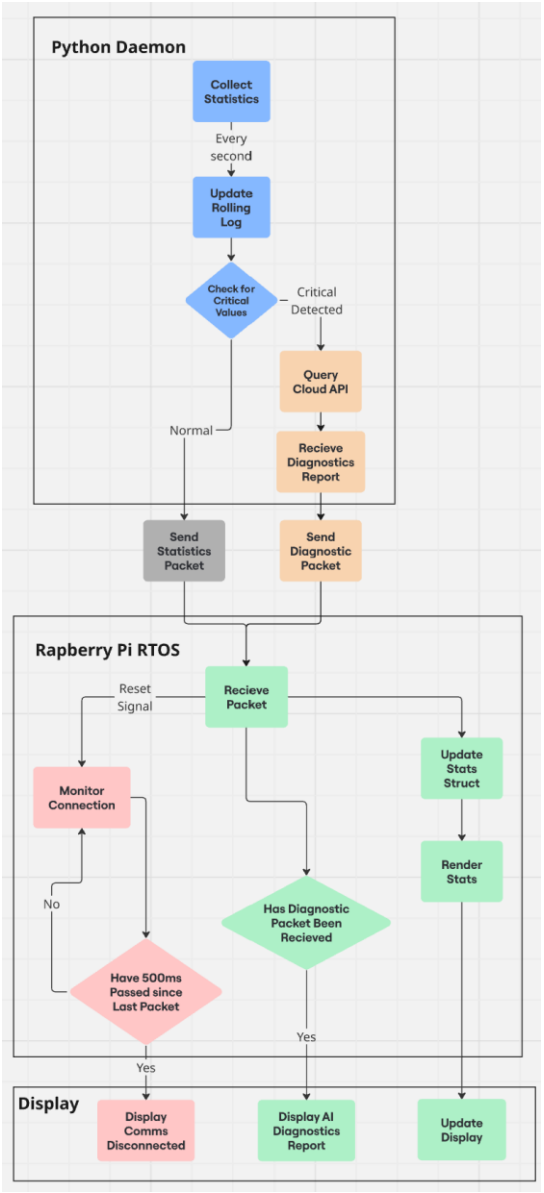


Fig. 1. Flowchart of the system architecture.

C. System Data Flow Graph

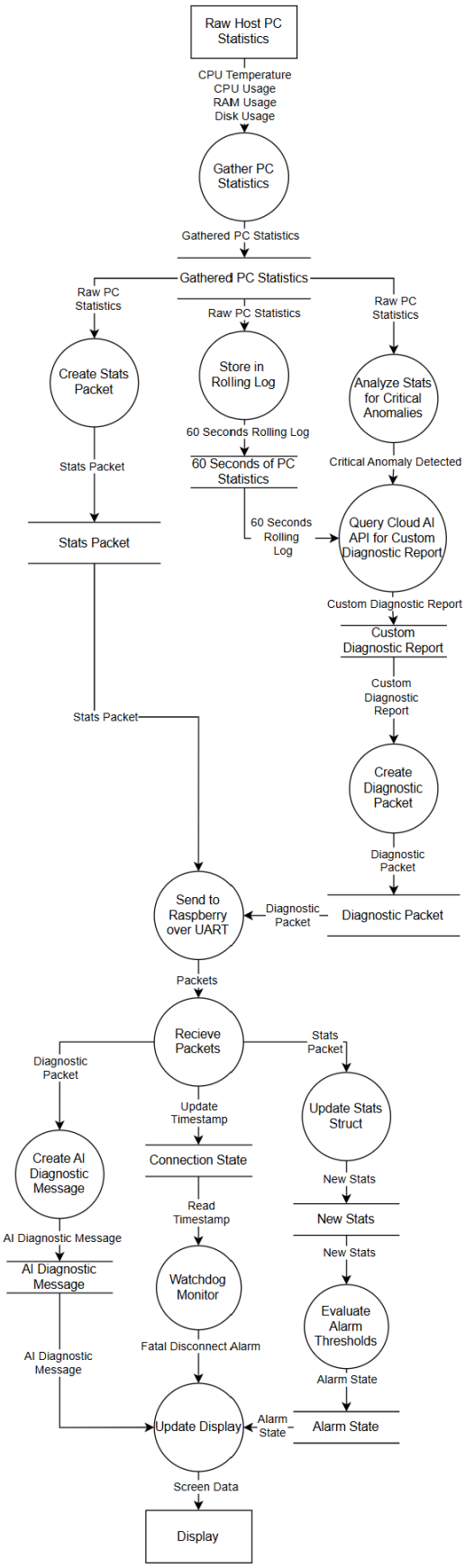


Fig. 2. Data Flow of the system architecture (Vertical because of space limitation).

D. Arcitectural Goals & Constraints

TABLE V. ARCHITECTURAL GOALS

Goal	Description
Host Independence	The Pi retains and displays the last known system state if the host connection is lost.
Real-Time Responsiveness	Display updates and alarm evaluations must meet hard timing deadlines (SRS-001, SRS-018).
Fault Visibility	Communication failures and threshold violations are surfaced immediately to the operator.
AI Augmentation	On-demand AI diagnostic reporting must not block normal monitoring (SRS-003).

TABLE VI. ARCHITECTURAL CONSTRAINTS

Constraint	Impact
Bare-metal Pi execution	No OS scheduler. Timing and concurrency managed via interrupts and a free-running clock.
UART bandwidth (115,200 baud)	Limits achievable display refresh rate when Pi is driven solely by host packets.
255-character AI report limit	AI diagnostic text must be truncated to fit the PCStats structure and UART packet.
Windows-only host	Host daemon libraries (wmi, pyserial) constrain the daemon to Windows platforms.

E. State Diagram

The system operates in exactly two top-level modes. Transitions are event-driven: a valid UART stats packet triggers entry into Monitoring Mode. Then a watchdog timeout triggers return to Standby Mode. An AI Report overlay is a temporary sub-state within Monitoring Mode that auto-dismisses after 10 seconds.

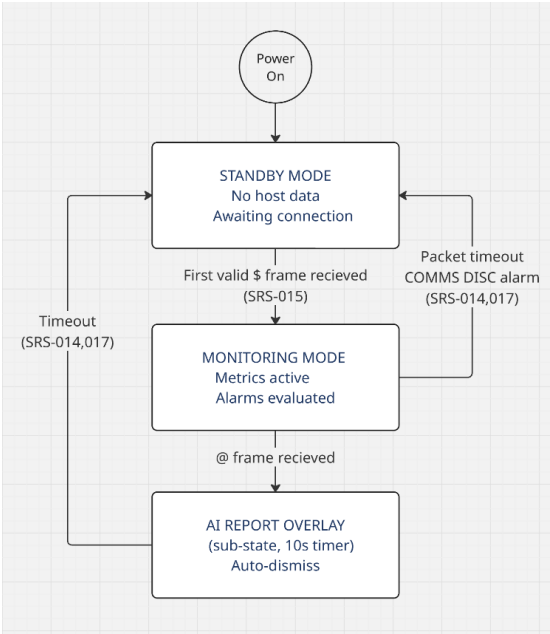


Fig. 3. State Diagram.

TABLE VII. STATE VARIABLES

Variable	Type	Owner Component	Description
isConnected	Int	Alarm / Mode	1 = Monitoring, 0 = Standby or disconnected
showingAIScreen	int	Display	1 = AI overlay active
lastPacketTime	uint32	Hardware Abstraction	SYSTEM_TIMER _CLO value at last valid $ packet
aiScreenTimer	uint32	Display	SYSTEM_TIMER _CLO value when AI overlay started
hasScreenBeenUpdated	int	Display	Dirty flag to suppress redundant framebuffer redraws

F. System Architecture & Components

1) Component – Host Daemon

a) Description

The Host Daemon is a Python application running on the Windows host PC. It serves as the sole source of system performance stats for the entire monitoring system. The daemon continuously polls OS-level hardware data, detects anomalous conditions, interfaces with the external AI API, and streams all data to the Raspberry Pi over UART. It maintains a 60-entry rolling log of all stats to provide context for AI-generated diagnostic reports.

b) Node

Host PC (Windows) | Source file: hostMonitor.py

c) Key Responsibilities

- Poll CPU usage, CPU temperature, RAM usage, disk usage, and the top 3 CPU-consuming processes at approximately 1-second intervals.
- Maintain a 60-entry rolling stat log for AI context.
- Detect anomalous stats conditions and invoke the AI API.
- Encode all data into structured ASCII UART packets and transmit to the Raspberry Pi.

2) Component – Communication Reception & Decoding

a) Description

This component receives raw byte packets from the host daemon over the UART hardware interface and reconstructs structured stats data. It acts as the boundary between the UART transport layer and all higher-level Pi application logic. An interrupt-driven ring buffer decouples byte delivery from the main kernel, ensuring no data is lost regardless of main loop timing.

Once a complete packet is assembled, the component identifies the packet type by its prefix character and writes parsed values into the shared PCStats data structure. Setting the isConnected flag on receipt of a valid metric frame also serves as the trigger for transitioning into Monitoring Mode.

b) Node

Raspberry Pi (Bare-Metal) | Source files: ringBuffer.c, kernel.c

c) Key Responsibilities

• Accept UART bytes via hardware interrupt and enqueue them into a circular ring buffer without data loss.
• Drain the ring buffer in the main loop and assemble complete line-terminated frames.
• Distinguish between metric data frames ($) and AI report frames (@) by prefix character.
• Parse metric frames and populate all fields of the shared PCStats struct.
• Set isConnected = 1 and update lastPacketTime upon receipt of a valid metric frame.

3) Component – Alarm Evaluation & Mode Management

a) Description

This component applies the rules that transform raw stats values from PCStats into visible alarms. It also controls which of the two system operating modes - Standby Mode or Monitoring Mode - is currently active. Evaluation runs once per main loop iteration and always selects the single highest-priority active alarm for display, ensuring critical conditions always supersede lower-severity ones.

The COMMS DISCONNECT alarm holds the highest priority and preempts all stat-based alarms whenever the host connection is lost. Mode transitions are driven by the isConnected state variable, which is set by the Communication Reception component on valid packet receipt and cleared by the Watchdog on timeout.

b) Node

Raspberry Pi (Bare-Metal) | Source files: kernel.c, globals.h

c) Key Responsibilities

• Compare each stat in PCStats against warning and critical thresholds each loop iteration.
• Select and surface only the highest-priority active alarm to the Display component.
• Manage the isConnected state variable to control Standby / Monitoring mode transitions.
• Trigger COMMS DISCONNECT alarm when the watchdog detects packet loss.

4) Component – Display & Rendering

a) Description

This component translates the current system state - as represented by the PCStats struct, active alarm, and connection status - into pixels on the 640x480 LCD framebuffer. The display subsystem has no decision-making authority. It exclusively renders what it is given by the alarm evaluation and communication components.

Three distinct rendering contexts exist: the Monitoring Mode layout (stats, progress bars, alarm banner, process list), the AI Report overlay (full-screen with auto-dismiss after 10 seconds), and the Standby Mode screen. The GPU framebuffer is allocated at boot via the ARM mailbox property channel.

b) Node

Raspberry Pi (Bare-Metal) | Source files: render.c, render.h, kernel.c

c) Key Responsibilities

• Render the standard Monitoring Mode layout: metrics, progress bars, alarm banner, and top-3 process list.
• Render the full-screen AI report overlay with wrapped text and a 10-second countdown auto-dismiss.
• Render the Standby Mode screen when isConnected = 0.
• Allocate and manage the GPU framebuffer via ARM mailbox at system initialization.

5) Component – Hardware Abstraction & Watchdog

a) Description

This component provides the foundational platform on which all other Pi subsystems execute. It initializes the ARM execution environment at boot, provides memory-mapped register definitions for all peripherals, defines the shared PCStats data structure, and manages the interrupt architecture for UART reception. The Watchdog function monitors the elapsed time since the last valid stats packet was received. If this exceeds the timeout threshold, the system transitions to Standby Mode and surfaces the COMMS DISCONNECT alarm.

Note: The current watchdog implementation uses a software timeout of 5 seconds and sets a disconnected flag. It does not issue a hardware reset. This is a divergence from SRS-018, which requires a hardware reset within 500 ms. Implementation requires use of the BCM2835 PM_WDOG register or an external watchdog IC.

b) Node

Raspberry Pi (Bare-Metal) | Source files: globals.h, start.S, kernel.c

c) Key Responsibilities

• Initialize ARM exception vectors, SVC and IRQ mode stacks at boot (start.S).
• Define all memory-mapped peripheral register addresses (globals.h).
• Define and own the shared PCStats data structure used by all Pi components.
• Route hardware IRQs to the appropriate ISR via irqDispatcher().
• Monitor lastPacketTime against the timeout threshold and assert isConnected = 0 on expiry.

6) Component – AI Report Integration

a) Description

This component manages all interaction with the external Google Gemini AI API. It is a sub-function of the Host Daemon and executes entirely on the host PC. The component is isolated here because it has distinct trigger conditions (anomaly detection) and a 60-second cooldown to prevent API saturation.

When triggered, the component serializes the 60-entry rolling stat log to JSON, constructs a prompt instructing the AI to provide root-cause analysis and hardware recommendations, POSTs the request to the Gemini API, and forwards the truncated response to the Pi as a separate UART AI Report packet. The Pi then displays this report as a full-screen overlay.

b) Node

Host PC (Windows) | Source file: hostMonitor.py

c) Key Responsibilities

• Monitor rolling metrics for anomaly conditions and trigger AI report generation with a cooldown.
• Serialize the 60-entry rolling log to JSON and construct the Gemini API request payload.
• POST the request to the Gemini API over HTTPS and receive the diagnostic response.
• Truncate the response to 255 characters and transmit as an @ UART frame to the Pi.

IV. Implementation (Waterfall Phase 3)

A. Overview

The implementation of the Hardware Monitor is divided into the host-side stats collection and the bare-metal Pi kernel. Due to the exhaustive nature of the component-level documentation, the full Software Detail Design (SDD) documents and complete source code have been omitted from this report for brevity.

B. Implementation - Host Daemon

This section details the implementation of the host daemon module **(hostMonitor.py). hostMonitor.py** runs on Windows under Python 3 and is the sole data source for the Raspberry Pi display. The daemon polls operating system performance stats at approximately one-second intervals using the psutil and wmi libraries, maintains a 60-entry rolling log history, detects metric anomalies that may warrant AI analysis, calls the Google Gemini 2.5 Flash API with the rolling log as context, and transmits structured ASCII data packets and AI report packets to the Pi over a serial (UART) connection. The module contains two primary functions: get_ai_report() and get_top_tasks(). The main execution body runs as a top-level while loop rather than a function.

Function: get_ai_report()
Generates an AI diagnostic report by serializing the rolling stat log history to JSON and submitting it to the Google Gemini 2.5 Flash API. Returns the model's response text, truncated and cleaned for transmission as a UART '@' packet. Includes a validity check for the API key and error handling for network or API failures.

Function: get_top_tasks()
Polls all running processes on the Windows host using psutil, aggregates CPU and memory usage by process name, filters out known system and virtual machine idle processes, and returns the three processes with the highest CPU usage. The raw per-process CPU percentages returned by psutil are normalized by the logical CPU core count to produce a per-core-equivalent percentage.

C. Implementation – RTOS Kernel

This section details the implementation of the main kernel module **(kernel.c). kernel.c** is the primary source file for the Pi firmware and contains the system entry point, hardware initialization sequence, infinite main control loop, UART packet assembly and parsing, alarm evaluation logic, display rendering, GPU mailbox interface, ARM hardware interrupt service routines, and the watchdog time. The module contains twenty functions from standard C library replacements (required due to the bare-metal environment), display control, hardware drivers, packet decoding, and the main execution loop. All functions are implemented without any OS support.

Function: memset()
Bare-metal replacement for the standard C library memset(). Sets each byte in a specified memory region to the given value. Required because the bare-metal build environment does not link against libc.

Function: str_cpy()
Bare-metal replacement for the standard C library strcpy(). Copies a null-terminated string from src to dest, including the null terminator. Used to copy task names and AI report text between PCStats structures.

Function: str_cmp()
Bare-metal replacement for the standard C library strcmp(). Performs a string comparison of two null-terminated strings. Used in the main loop to detect changes in task names between successive PCStats updates.

Function: memcpy()
Bare-metal replacement for the standard C library memcpy(). Copies n bytes from the src memory region to the dest memory region byte by byte. The source and destination regions must not overlap.

Function: drawDisplay()
Renders the complete real-time monitoring display to the framebuffer. The function contains a color-coded connection status header, three metric sections (CPU, RAM, Disk) each with a progress bar and numeric readouts, a priority-ordered alarm banner, and a top three resource-consuming process list. Alarm evaluation is performed inline by testing PCStats values against threshold constants in descending priority order (FATAL > CRITICAL > WARNING > NOMINAL).

Function: drawAIDisplay()
Renders a full-screen AI diagnostic report overlay in the lower portion of the display. The overlay consists of a red bordered box containing the AI report text word-wrapped at 65 characters per line, a title header, and a static countdown message at the bottom of the box.

Function: uart_putc()
Transmits a single byte over the UART0 transmit path. Busy-waits by polling the UART flag register until the transmit FIFO is not full, then writes the byte to the UART data register. Used during initialization for debug output.

Function: uart_puts()
Transmits a null-terminated string over UART0 by calling uart_putc() for each character in sequence until the null terminator is reached. Used during kernel initialization to send debug status messages.

Function: mboxWrite()
Writes a 28-bit data value tagged with a 4-bit channel number to the Raspberry Pi GPU property mailbox. Waits until the mailbox write register indicates it is not full before performing the write.

Function: mboxRead()
Reads a response message from the GPU property mailbox on the specified channel. Discards any mailbox responses that do not match the requested channel number, continuing to poll until a matching response arrives.

Function: parseString()
Extracts a string token from a UART packet buffer starting at the position indicated by the caller's index. Copies characters into the destination buffer until a field delimiter (','), newline, carriage return, or null terminator is encountered. Advances the caller's index past all consumed characters so subsequent parsing can continue from the correct position.

Function: getSystemTime()
Reads the current value of the BCM2835 free-running 1 MHz system timer (TIMER_CLO, the lower 32 bits of the 64-bit counter). Used by the watchdog monitor and the AI overlay auto-dismiss timer to measure elapsed time in microseconds.

Function: dummyDelay()
Provides a coarse software busy-wait delay by executing a volatile counting loop. The actual elapsed time depends on CPU clock frequency and pipeline conditions. Used for short, non-critical timing gaps during hardware initialization where a precisely calibrated delay is not required.

Function: parseNum()
Extracts a non-negative decimal integer from a UART packet buffer at the current index position. Skips any leading non-digit characters (such as the comma delimiter from the preceding token) before accumulating digit characters into the result. Advances the caller's index past all consumed characters.

Function: decodeStatsString()
Parses a complete UART frame into the shared PCStats structure. Handles both the '$' metric data frame type and the '@' AI report frame type. For data frames, iterates through the payload tokenizing on comma delimiters and dispatching each two-character key tag to the corresponding PCStats field via parseNum() or parseString(). For AI report frames, copies the report payload directly into the m_aiReport field.

Function: checkUartBuffer()
Drains the ring buffer byte by byte and assembles complete line-delimited UART packets. ARM IRQs are briefly disabled around each rbPop() call to ensure atomic access to the shared ring buffer indices. When a complete packet is detected (terminated by '\n' or '\r'), calls decodeStatsString() to parse it into the PCStats structure.

Function: c_uartIRQHandler()
UART0 interrupt service routine. Called by c_irqDispatcher() when IRQ_PENDING2 bit 25 is asserted. Immediately clears all UART interrupt flags to acknowledge the interrupt, clears any receive error status, then drains all available bytes from the UART0 RX hardware FIFO into the ring buffer.

Function: c_irqDispatcher()
Top-level ARM IRQ dispatcher called from the IRQ exception vector. Tests the interrupt pending register to determine which peripheral triggered the interrupt and routes execution to the appropriate handler. Currently handles only the UART0 interrupt (IRQ_PENDING2 bit 25) and is structured for straightforward extension to additional interrupt sources.

Function: enableUartInterrupts()
Configures the UART0 peripheral and ARM generic interrupt controller to generate IRQ exceptions on UART receive events, then globally enables ARM IRQ exceptions by clearing the CPSR I-bit. Also purges any pre-existing receive errors and stale RX FIFO data before enabling interrupts to start with a clean state.

Function: kernel_main()
System entry point called unconditionally from the startup assembly module (start.S) after ARM execution mode and stack initialization. Performs all one-time hardware initialization and then executes the infinite polling main loop for the lifetime of the system. The main loop coordinates UART packet reception, watchdog timeout detection, AI overlay timer management, state change detection, and framebuffer rendering.

D. Implementation – Ring Buffer

This section details the implementation of the ring buffer module (**ringBuffer.c** / **ringBuffer.h**). The ring buffer is a 1,024-byte circular buffer that serves as the ISR to main loop communication bridge for UART received data. The UART0 interrupt service routine pushes received bytes into the buffer as they arrive from the host. The main processing loop pops bytes during packet assembly. This producer-consumer decoupling prevents byte loss at 115,200 baud regardless of main loop execution latency. The module

contains four functions that constitute the complete ring buffer.

Function: rbInit()
Initializes the ring buffer by resetting both the head (write) and tail (read) index variables to zero. Must be called once during system startup, before UART interrupts are enabled, to ensure the buffer starts in a defined empty state.

Function: rbPush()
Enqueues a single byte into the ring buffer. This function is called exclusively from within the UART interrupt service routine and must complete quickly without blocking. If the buffer is full (only RING_BUFFER_SIZE - 1 bytes may be stored at one time), the incoming byte is silently discarded to prevent the ISR from stalling.

Function: rbPop()
Dequeues a single byte from the ring buffer and writes it to the caller-provided output pointer. Called by the main loop during packet assembly. Returns immediately with a failure code if no data is available, allowing the main loop to continue without blocking.

Function: rbIsEmpty()
Returns whether the ring buffer currently contains any unread bytes. Provides a lightweight check that can be used before attempting a pop operation. The empty condition is defined as head and tail pointing to the same buffer slot.

E. Implementation – Display & Rendering

This section details the implementation of the display rendering module (**render.c** / **render.h**). **render.c** provides the complete implementation responsible for writing pixels, shapes, characters, strings, and progress bars to the 640x480 LCD display on the Raspberry Pi. All rendering operations write directly to the GPU-allocated framebuffer using 32-bit ARGB color encoding (stored in A-B-G-R byte order in memory). The module contains seven functions that form the entire display primitive library consumed by kernel.c to produce the real-time monitoring UI.

Function: drawPixel()
Writes a single 32-bit ARGB pixel to the framebuffer at the specified (x, y) screen coordinate. Performs a bounds check against the 640x480 display dimensions before writing to prevent out-of-bounds memory corruption.

Function: drawRect()
Draws a solid filled rectangle on the framebuffer. Iterates over every pixel in the defined region, calling drawPixel() for each. Clipping is handled implicitly by drawPixel()'s bounds check.

Function: drawChar()
Renders a single 8x8 monospace bitmap character from the font8x8 lookup table onto the framebuffer at the specified position. Each of the 8 rows of the character glyph is read as a bitmask. Set bits render as the foreground color and clear bits render as the background color. Characters outside the printable ASCII range (32-126) are silently ignored.

Function: drawString()
Renders a null-terminated ASCII string to the framebuffer by calling drawChar() for each character in sequence. The x position is advanced by 8 pixels (one character width) after each character, producing a left-to-right horizontal layout.

Function: drawProgressBar()
Draws a horizontal progress bar composed of two layered filled rectangles. A full-width background rectangle is drawn first, then a proportionally shorter foreground rectangle is drawn over it to represent the current fill level. The percent value is clamped to the range 0-100 before use.

Function: intToString()
Converts a signed integer to its null-terminated decimal ASCII string representation, storing the result in the caller-provided output buffer. Handles zero, positive, and negative values. The output buffer must be at least 12 bytes to safely accommodate any 32-bit integer value.

Function: drawNumber()
Renders a signed integer as a decimal string on the framebuffer at the specified coordinates. Internally converts the integer to a string using intToString() and renders it using drawString(). Returns the pixel x coordinate immediately following the last rendered digit, enabling the caller to continue rendering additional text on the same baseline.

V. TESTING & RESULTS (WATERFALL PHASE 4)

A. Test Cases

The software system test cases are individual vectors for testing the SRS. Each test case has a set of inputs as well as expected results to verify the fulfillment of the tested software requirement.

TABLE VIII. TEST CASES

SRS ID	SSTC ID	Input	Expected Results
SRS-001	SSTC-001	PC Monitor is actively monitoring	The following information is updated once every 20ms: • Monitored Temperature • Monitored CPU usage • Monitored Memory usage • Monitored Disk usage
SRS-002	SSTC-002	PC Monitor is in standby	The following categories of information are listed along with a value for each: • Monitored temperature • Monitored CPU usage • Monitored Memory usage • Monitored Disk usage
SRS-003	SSTC-003	AI analysis of log is requested	Current log is formatted as JSON and sent over AI API
SRS-004	SSTC-004	AI analyzed log is received over AI API	AI analyzed log is displayed
	SSTC-005	PC Monitor is monitoring	The following categories of information are listed along with a value for each: • Monitored temperature • Monitored CPU usage • Monitored Memory usage • Monitored Disk usage
SRS-006	SSTC-006	CPU % in use is 75%	CPU HIGH WARNING alarm is not active
	SSTC-007	CPU % in use is 76%	CPU HIGH WARNING alarm is triggered
	SSTC-008	CPU % in use is 95%	CPU HIGH WARNING alarm is active
SRS-007	SSTC-009	RAM % in use is 85%	RAM HIGH WARNING alarm is not active
	SSTC-010	RAM % in use is 86%	RAM HIGH WARNING alarm is triggered
	SSTC-011	RAM % in use is 95%	RAM HIGH WARNING alarm is active
SRS-008	SSTC-012	Disk % in use is 70%	DISK HIGH WARNING alarm is not active
	SSTC-013	Disk % in use is 71%	DISK HIGH WARNING alarm is triggered
	SSTC-014	Disk % in use is 90%	DISK HIGH WARNING alarm is active
SRS-009	SSTC-015	PC Temperature is 80 °C	TEMP HIGH WARNING alarm is not active
	SSTC-016	PC Temperature is 81 °C	TEMP HIGH WARNING alarm is triggered
	SSTC-017	PC Temperature is 95 °C	TEMP HIGH WARNING alarm is active
SRS-010	SSTC-018	CPU % in use is 95%	CPU CRITICAL WARNING alarm is not active
	SSTC-019	CPU % in use is 96%	CPU CRITICAL WARNING alarm is triggered
	SSTC-020	CPU % in use is 100%	CPU CRITICAL WARNING alarm is active
SRS-011	SSTC-021	RAM % in use is 95%	RAM CRITICAL WARNING alarm is not active
	SSTC-022	RAM % in use is 96%	RAM CRITICAL WARNING alarm is triggered
	SSTC-023	RAM % in use is 100%	RAM CRITICAL WARNING alarm is active
SRS-012	SSTC-024	Disk % in use is 90%	DISK CRITICAL WARNING alarm is not active
	SSTC-025	Disk % in use is 91%	DISK CRITICAL WARNING alarm is triggered
	SSTC-026	Disk % in use is 90%	DISK CRITICAL WARNING alarm is active
SRS-013	SSTC-027	PC Temperature is 95 °C	TEMP CRITICAL WARNING alarm is not active
	SSTC-028	PC Temperature is 96 °C	TEMP CRITICAL WARNING alarm is triggered
	SSTC-029	PC Temperature is 100 °C	TEMP CRITICAL WARNING alarm is active
SRS-014	SSTC-030	Communication is lost with the PC Monitor	COMMS DISCONNECT alarm is triggered
SRS-015	SSTC-031	PC Monitor is in standby A connection is detected over UART	PC Monitor enters Monitoring mode
SRS-017	SSTC-032	PC Monitor is powered on	PC Monitor enters Standby mode
	SSTC-033	PC Monitor is disconnected from monitored device	PC Monitor enters Standby mode
SRS-018	SSTC-034	Watchdog timer is not serviced for 500ms	PC Monitor system resets
	SSTC-035	Watchdog timer is serviced after 499ms	PC Monitor system does not reset
SRS-019	SSTC-036	CPU CRITICAL WARNING is triggered	AI analyzed log is generated and displayed addressing the PC status relating to the CPU
	SSTC-037	RAM CRITICAL WARNING is triggered	AI analyzed log is generated and displayed addressing the PC status relating to the RAM
	SSTC-038	DISK CRITICAL WARNING is triggered	AI analyzed log is generated and displayed addressing the PC status relating to the Disk
	SSTC-039	TEMP CRITICAL WARNING is triggered	AI analyzed log is generated and displayed addressing the PC status relating to the Temperature

B. Requirements Traceability Matrix

The table below traces each SRS requirement to the responsible architectural component and records implementation status. Full function-level traceability is provided in the Implementation Section.

TABLE IX. REQUIREMENTS TRACEABILITY MATRIX

SRS ID	Requirement Summary	Responsible Component	Status
SRS -001	Display update every 20 ms	Display & Rendering, Host Daemon	PARTIAL - limited by ~1 Hz host transmit rate
SRS -002	Display temp, CPU, RAM, disk	Display & Rendering	Implemented
SRS -003	AI log formatted in JSON	AI Report Integration	PARTIAL - The user can't select to generate a report.
SRS -004	Display AI analyzed log	Display & Rendering	Implemented
SRS -005	Capture data over UART	Communication Reception & Decoding	Implemented
SRS -006	CPU HIGH WARNING > 75%	Alarm Evaluation & Mode Management	Implemented
SRS -007	RAM HIGH WARNING > 85%	Alarm Evaluation & Mode Management	Implemented
SRS -008	DISK HIGH WARNING > 70%	Alarm Evaluation & Mode Management	Implemented
SRS -009	TEMP HIGH WARNING > 80 °C	Alarm Evaluation & Mode Management	Implemented
SRS -010	CPU CRITICAL > 95%	Alarm Evaluation & Mode Management	Implemented
SRS -011	RAM CRITICAL > 95%	Alarm Evaluation & Mode Management	Implemented
SRS -012	DISK CRITICAL > 90%	Alarm Evaluation & Mode Management	Implemented
SRS -013	TEMP CRITICAL > 95 °C	Alarm Evaluation & Mode Management	Implemented
SRS -014	COMMS DISCONNECT alarm	Alarm Evaluation, Hardware Abstraction & Watchdog	Implemented
SRS -015	Enter Monitoring Mode on UART	Alarm Evaluation & Mode Management	Implemented
SRS -017	Enter Standby on power-up/disconnect	Alarm Evaluation, Hardware Abstraction & Watchdog	Implemented
SRS -018	Watchdog reset within 500 ms	Hardware Abstraction & Watchdog	DEVIATION - 5s timeout. No hardware reset
SRS -019	User option to generate AI log	AI Report Integration, Display & Rendering	DEVIATION - host-triggered, no user input
SRS -020	Flash write within 10 ms	(No component assigned)	GAP - not implemented
SRS -021	AI API uses TLS 1.3	AI Report Integration	PARTIAL - TLS version not explicitly pinned
SRS -022	Watchdog uses independent clock	Hardware Abstraction & Watchdog	Implemented
SRS -023	Full operation within 500 ms post-reset	Hardware Abstraction & Watchdog	GAP - no hardware reset to recover from

C. Known Gaps & Deviations

1) Watchdog Timer (SRS-018, SRS-022, SRS-023)

Requirement: Hardware watchdog must reset the system within 500 ms of non-service. System must be fully operational within 500 ms post-reset.
Current State: A software timer detects packet loss after 5 seconds and sets a disconnected flag, but does not trigger a hardware reset.
Impact: The 500 ms hard deadline and post-reset recovery requirement are both unmet. Implementation requires the BCM2835 hardware watchdog peripheral (PM_WDOG register) or an external watchdog IC.

2) Display Update Rate (SRS-001)

Requirement: Metrics updated every 20 ms (50 Hz).
Current State: The Pi renders once per received packet. The host transmits approximately once per second, giving an effective rate of approximately 1 Hz.
Impact: The hard 20 ms deadline is not met. Resolution requires either increasing the host transmit rate to 50 Hz or implementing independent Pi-side redraw using cached data at 20 ms intervals.

3) TLS 1.3 (SRS-021)

Requirement: AI API communication shall use TLS 1.3.
Current State: The Python requests library relies on the system OpenSSL, which supports TLS 1.3 but does not enforce it as a minimum version.
Impact: A TLS 1.2 downgrade is possible on older hosts. Resolution requires explicit version pinning via ssl.SSLContext with TLSVersion.TLSv1_3.

VI. USER MANUAL & DOCUMENTATION

This section provides step-by-step instructions for setting up and operating the Hardware RTOS PC Monitor system. The system monitors live PC performance metrics on a host Windows machine and displays them on a Raspberry Pi 2B running a bare-metal real-time operating system (RTOS), emulated via QEMU.

The system consists of two software components:
• kernel.elf: the bare-metal RTOS firmware running on the QEMU-emulated Raspberry Pi 2B.
• hostMonitor.py: a Python daemon running on the host PC that collects and transmits system metrics.

NOTE: This manual uses the Windows Subsystem for Linux (WSL) with Ubuntu to build and emulate the firmware. All terminal commands in this document are run inside the WSL Ubuntu environment unless otherwise stated.

A. Prerequisites

1) Host PC Requirements

- Operating System: Windows 10 or Windows 11.
- Visual Studio Code (VS Code) installed on Windows.
- Internet connection for downloading packages.

2) Python Requirements (for hostMonitor.py)

The following Python packages are required to run the host monitor daemon. These are installed on the Windows host (not inside WSL):

- psutil — CPU, RAM, and disk usage collection.
- wmi — Windows thermal and hardware sensor readings.
- pyserial — UART/serial port communication to the QEMU virtual port.
- requests — library for making HTTP requests for the AI diagnostic reports.

B. Environment Setup

Follow the steps below in order. Each step must be completed before proceeding to the next.

1) Step 1 — Install WSL

Open a Windows Command Prompt or PowerShell as Administrator and run:

```
wsl --install
```

When prompted, restart your PC to complete the WSL installation.

2) Step 2 — Install Ubuntu

After restarting, open Command Prompt or PowerShell again and run:

```
wsl --install -d Ubuntu
```

Follow the on-screen prompts to create a UNIX username and password. Once complete, your Ubuntu terminal prompt will appear in green, indicating a successful installation.

3) Step 3 — Update Package Lists

Inside the Ubuntu terminal, update all package lists to ensure you receive the latest versions:

```
sudo apt update
```

4) Step 4 — Install the Compiler and Emulator

Install the ARM cross-compiler, binary utilities, build tools, and QEMU system emulator in a single command:

```
sudo apt install gcc-arm-none-eabi binutils-arm-none-eabi make qemu-system-arm -y
```

This installs the following tools:

TABLE X. EMULATOR TOOLS

Package	Purpose
gcc-arm-none-eabi	ARM cross-compiler for bare-metal firmware
binutils-arm-none-eabi	ARM binary utilities (linker, assembler)
make	Build automation tool
qemu-system-arm	ARM system emulator (emulates Raspberry Pi 2B)

5) Step 5 — Create the Project Workspace

In the Ubuntu terminal, navigate to your home directory and create the project folder:

```
cd ~
mkdir HW_RTOS
cd HW_RTOS
```

This folder will contain all project source files.

6) Step 6 — Connect VS Code to WSL

1. Open VS Code on Windows.
2. Open the Extensions panel (Ctrl+Shift+X) and search for WSL.
3. Install the WSL extension published by Microsoft.
4. Once installed, click Connect to WSL in the notification or press F1 and select WSL: Connect to WSL.
5. After VS Code reloads, confirm the blue WSL: Ubuntu status box appears in the bottom-left corner of the window.
6. In VS Code, go to File > Open Folder and navigate to the HW_RTOS folder created in Step 5.

NOTE: All subsequent terminal commands in VS Code must be run inside the WSL Ubuntu terminal (Terminal > New Terminal). Verify that the terminal prompt shows your Ubuntu username, not a Windows path.

7) Step 7 — Add Project Source Files

Place all project source files into the HW_RTOS folder you opened in VS Code. The required files are:

- kernel.c — Main RTOS kernel
- render.c / render.h — GPU framebuffer rendering
- ringBuffer.c / ringBuffer.h — UART ring buffer
- globals.h — Hardware register definitions and global constants
- font.h — Bitmap font data
- Makefile — Build configuration
- hostMonitor.py — Host PC monitoring daemon

C. Building the Firmware

With all source files in place, build the firmware from the WSL Ubuntu terminal inside the HW_RTOS directory:

```
make
```

If the build is successful, a kernel.elf file will be generated in the project directory. This is the compiled bare-metal firmware image that will be loaded into the QEMU emulator.

NOTE: If the build fails, verify that all source files are present in the HW_RTOS directory and that the ARM cross-compiler was installed correctly in Step 4.

D. Running the QEMU Emulator

The QEMU emulator simulates a Raspberry Pi 2B and runs the compiled kernel firmware. The emulator exposes a virtual serial (UART) port over TCP, which the host monitor uses to transmit metrics.

1) Launch the Emulator

From the WSL Ubuntu terminal in the HW_RTOS directory, run the following command:

```
qemu-system-arm -M raspi2b -m 1G -serial tcp:127.0.0.1:4321,server,nowait,nodelay -kernel kernel.elf
```

The QEMU emulator window will open and display the Raspberry Pi framebuffer output at 640x480 resolution. The system will show the monitoring display, initially in Standby Mode until a host connection is received.

2) QEMU Command Parameters

TABLE XI. QEMU COMMAND PARAMETERS

Parameter	Description
-M raspi2b	Emulate a Raspberry Pi 2B machine.
-m 1G	Allocate 1 GB of emulated RAM.
-serial tcp:127.0.0.1:4321,...	Expose UART0 as a TCP server on localhost port 4321. nowait means QEMU will not block waiting for a client connection before booting.
-kernel kernel.elf	Load and execute the compiled bare-metal firmware.

NOTE: The QEMU window must remain open for the duration of system operation. Closing the QEMU window will terminate the emulated Raspberry Pi.

E. Running the Host Monitor

Once the QEMU emulator is running, launch the host monitor daemon on the Windows host. The host monitor collects PC metrics and transmits them to the QEMU virtual serial port.

1) Install Python Dependencies

Open a Windows Command Prompt (not WSL) and install the required Python packages:

```
pip install psutil wmi pyserial requests
```

2) Configure the Serial Port

The hostMonitor.py script must be configured to connect to the QEMU virtual serial port. Open hostMonitor.py and verify the serial connection parameters match the following:

- Host: 127.0.0.1
- Port: 4321
- Baud Rate: 115200
- Format: 8N1 (8 data bits, no parity, 1 stop bit)

3) Configure the AI Report

The hostMonitor.py script must be configured to be able to communicate with Google's Gemini LLM. More specifically a Google Gemini API Key needs to be generated from Google and imported in the API_KEY variable in hostMonitor.py

4) Launch the Host Monitor

From a Windows Command Prompt in the project directory, run:

```
python hostMonitor.py
```

The host monitor will begin collecting PC metrics every 1 second and transmitting them to the QEMU emulator over the TCP serial connection. The QEMU display window will transition from Standby Mode to Monitoring Mode and begin displaying live statistics

F. System Operation

Once both the QEMU emulator and hostMonitor.py are running, the system will display the following information on the QEMU window:

- CPU Usage (%): with color-coded progress bar
- RAM Usage (%): with color-coded progress bar
- Disk Usage (%): with color-coded progress bar
- CPU Temperature (°C): with color-coded indicator
- Top 3 active processes by CPU consumption
- Active alarm banners when thresholds are exceeded.

1) System Modes

TABLE XII. SYSTEM MODES

Alarm	Threshold
Standby Mode	Entered on power-up or when the UART connection from the host is not detected. The display shows a waiting state. The last known metric values are retained.
Monitoring Mode	Entered when a UART connection is detected from the host. Metrics are updated every 20 ms and displayed in real time.

2) Alarm Thresholds

The system will display alarm banners on the QEMU screen when the following thresholds are exceeded:

TABLE XIII. ALARM THRESHOLDS

Alarm	Severity
CPU HIGH WARNING	Warning
CPU CRITICAL WARNING	Critical
RAM HIGH WARNING	Warning
RAM CRITICAL WARNING	Critical
DISK HIGH WARNING	Warning
DISK CRITICAL WARNING	Critical
TEMP HIGH WARNING	Warning
TEMP CRITICAL WARNING	Critical

G. Stopping the System

To shut down the system gracefully:

- In the Windows Command Prompt running hostMonitor.py, press Ctrl+C to terminate the host daemon.
- Close the QEMU emulator window, or return to the WSL terminal where QEMU is running and press Ctrl+C.

NOTE: The RTOS kernel on the Pi retains the last known metric values after the host connection is lost. The display will remain on the last received data until the emulator is closed or restarted.

VII. CONCLUSION & FUTURE WORK

This project successfully showed a reliable, hardware monitoring solution that is designed to survive host-system crashes. By decoupling the diagnostic display from the host operating system, we resolved the vulnerability of "telemetry collapse" inherent in traditional consumer-grade hardware monitors. The architecture uses a lightweight Python host daemon for data gathering and a bare-metal RTOS environment for independent visualization. Additionally, by querying Google's Gemini Large Language Model, the system successfully mirrors state of the art AIOps frameworks, proving that automated human free root cause analysis can be useful for consumer-grade PC infrastructure.

A. Future Work

1) Physical Hardware Implementation

The current bare-metal RTOS implementation was developed and tested using the QEMU emulator. Future iterations will focus on moving the implementation to physical Raspberry Pi hardware and resolving any timing or peripheral alignment issues between the emulator and the physical hardware.

2) Edge AI Integration

To remove cloud latency, external API dependencies and privacy concerns, future versions will transfer the intelligent reporting entirely on the hardware. By using a more powerful board (e.g Raspberry Pi 5), the system could run a Small Language Model such as Llama 3.2 locally. Also, we will move the rolling log from the host daemon to the PI's local memory, so the system can autonomously generate post-mortem crash reports using the surviving data buffer even after total host disconnection.

3) Seamless Host Daemon

The Python data extraction script requires manual execution and setup. Future versions will package the script as a native Windows Background Service, along with all its dependencies, allowing the daemon to boot automatically on startup and operate silently without user intervention.

4) Hardware Interactivity & On Demand Reporting

The current system lacks the ability for user input, failing the SRS-003 requirement. The addition of GPIO button interrupts would give users the ability to manually trigger AI report and generate an on-the-fly system diagnostic report at any time, without waiting for an automated anomaly threshold to be passed.

5) Power Management

Standby power consumption currently exceeds the originally planned 50-mA requirement. Future versions should implement CPU clock scaling and GPU peripheral power-down protocols when connection is lost.

B. Requirements to be Implemented

The following requirements are planned to be implemented in a future version of the software:

1) Functional Requirements

SRS-016:

The following information shall be logged every 50ms:

- Monitored Temperature
- Monitored CPU usage
- Monitored Memory usage
- Monitored Disk usage
- Internal Temperature
- Internal CPU usage
- Internal Memory usage
- Internal Disk usage

2) Non-Functional Requirements

SRS-020:

Logs shall be written to Flash within 10ms of being received.

SRS-024:

While in Standby Mode, the power consumed shall be less than 50mA.

C. Team Contributions

TABLE XIV. CONTRIBUTION BY TEAM MEMBER

Member	Contribution
Jason Kostopoulos	Development of bare metal kernel along with all modules for the Raspberry, development of Host Daemon, writing of initial project proposal, State of the Art and Literature Survey.
Ryan Chang	Initial project set up and ideation, development of small Python server program for the host daemon (later cut due to bare metal networking limitations), presentation, timing requirements.
Brad Bernia	Authoring of full waterfall documentation, including requirement analysis and elicitation (SRS Document), software architecture (SAD Document), Implementation (SDD Documents), testing and recording of demo.

REFERENCES

[1] G. Liu and T. Wood, "Cloud-Scale Application Performance Monitoring with SDN and NFV," 2015 IEEE International Conference on Cloud Engineering, Mar. 2015.

[2] Q. Zhang, Z. Chen, Y. Zhou, X. Ouyang, and R. Wang, "Monitoring and diagnosing performance issues in large-scale distributed systems," in Proceedings of the ACM SIGOPS Conference. ACM, 2020.

[3] F. Lin *et al.*, "Near-Realtime Server Reboot Monitoring and Root Cause Analysis in a Large-Scale System," *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume (DSN-S)*, pp. 37–40, Jun. 2021.

[4] M. Bidollahkhani, F. Nordsiek, and J. M. Kunkel, "When GPUs Fail Quietly: Observability-Aware Early Warning Beyond Numeric Telemetry," *arXiv.org*, 2026.

[5] J.-G. Lou, Q. Lin, and Q. Fu, "Anomaly detection in monitoring data: Challenges and opportunities," IEEE Transactions on Network and Service Management, vol. 19, no. 3, pp. 2765–2780, 2022.

[6] Dell Technologies, "Out-of-Band Management with Integrated Dell Remote Access Controller (iDRAC)," *Dell Engineering Whitepaper*, 2021.

[7] N. P. Tran, B. Jaumard, O. Delgado, T. Glatard, K. Premkumar, and K. Ni, "LLM-Augmented Knowledge Base Construction for Root Cause Analysis," *IEEE Access*, vol. 14, pp. 16802–16816, 2026.

[8] X. Luo, S. M. N. Jha, A. Sinha, Z. Li and Y. Liu, "ALPHA: LLM-Enabled Active Learning for Human-Free Network Anomaly Detection," *2025 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, Austin, TX, USA, 2025.

[9] Valvers Bare Metal Raspberry Pi Tutorial, https://www.valvers.com

[10] Baking Pi – Operating System Development, University of Cambridge, Department of Computer Science and Technology – Raspberry Pi: Baking Pi – Operating Systems Development